PROYECTO INTEGRADOR

PrestaLab

Sistema de reserva y control de préstamo de equipo y espacios de laboratorio

ENTREGA 2 — Análisis de requisitos y arquitectura

Carrera	Ingeniería en Software
Materia	Arquitectura Orientada a Servicios
Docente	Ing. Luis Leonardo Mendoza Reséndiz
Equipo	Tranquilo nos la navegamos
Integrantes	Jhonatan Uriel Morales Badillo, José Vidal López Salazar, David Sosa Gómez, Alejandro Ramírez Contreras
Periodo / Semana	Septiembre – Diciembre 2026 · Semana 2 de 8
Repositorio	https://github.com/JhonaRaida/prestalab · tag: entrega-2

Reporte breve de la entrega

Entrega	2 — Análisis de requisitos y arquitectura
Nombre del proyecto	PrestaLab (nombre provisional)
Integrantes	Jhonatan Uriel Morales Badillo, José Vidal López Salazar, David Sosa Gómez, Alejandro Ramírez Contreras
Objetivo de la semana	Convertir la propuesta en un diseño técnico inicial: qué debe hacer el sistema, qué cualidades debe cumplir y cómo se divide en servicios.
Qué se diseñó	10 requisitos funcionales y 9 no funcionales con criterios verificables, 4 roles con matriz de permisos, 10 entidades del dominio, catálogo de 5 servicios más un API Gateway, mapa de comunicación entre servicios, diagrama de arquitectura SOA y stack tecnológico justificado.
Qué cambió respecto a la entrega anterior	Se revisó la propuesta y se corrigieron 9 puntos (sección 1): se separó estado administrativo de disponibilidad, se definió qué es una sanción y el alcance de las notificaciones, y se tomaron las decisiones de tecnología que estaban abiertas.
Problemas encontrados	La propuesta mezclaba el estado de un recurso con su disponibilidad en un horario; no estaba claro quién debía evitar reservas empalmadas; había que dividir el sistema sin duplicar responsabilidades entre servicios; y las reglas de préstamo no tenían valores concretos.
Cómo se resolvieron	Inventario guarda solo el estado administrativo y Reservas calcula la disponibilidad. El traslape se bloquea en la base de datos con una restricción de PostgreSQL, además de la validación en código. Cada dato tiene un solo servicio dueño y los servicios se consultan solo por API. Se propusieron 9 reglas con valores configurables, que se validarán con un encargado de laboratorio.
Pendientes para la semana 3	Diseño de la API: recursos REST, mínimo 20 endpoints, contratos JSON, códigos HTTP y errores. Validar con un encargado de laboratorio las reglas de negocio de la sección 2.3.
Enlace al repositorio y versión	https://github.com/JhonaRaida/prestalab · tag: entrega-2

Resumen de la propuesta (Entrega 1)

PrestaLab busca resolver la falta de control en el préstamo de equipo y espacios de laboratorio: hoy se registra en cuadernos, hojas de cálculo y mensajes, lo que provoca

reservas empalmadas, falta de trazabilidad, devoluciones tardías sin seguimiento y ausencia de datos para decidir.

La solución es un sistema orientado a servicios que valida la disponibilidad antes de reservar, registra la entrega y la devolución de cada recurso, mantiene un historial y genera reportes de uso, retrasos e incidencias para alumnos, docentes, encargados de laboratorio y administradores.

1. Revisión de la propuesta de la Entrega 1

Antes de avanzar se revisó la propuesta anterior buscando ambigüedades que afectaran al diseño. Se encontraron los siguientes puntos y se corrigieron en este documento. El resto de la propuesta (contexto, problema, actores, objetivos y beneficios) se mantiene como en la Entrega 1.

1.1 Hallazgos y correcciones

#	Hallazgo en la Entrega 1	Corrección aplicada	Sección
1	El catálogo de inventario incluía «estado y disponibilidad», mezclando dos conceptos.	Inventario guarda solo el estado administrativo del recurso (activo, mantenimiento o baja). La disponibilidad en un horario la calcula Reservas.	2, 6
2	Se hablaba de «sanciones», pero el cobro de multas está fuera de alcance y no se definió qué es una sanción.	Sanción = bloqueo temporal automático para reservar por una devolución tardía. No hay cobro económico.	2.3, 6
3	Las notificaciones «básicas» no tenían canal definido.	Versión inicial: cada aviso se registra en base de datos y se envía por correo de prueba o simulado. Un fallo no bloquea la operación principal.	6, 7
4	Las 12 funciones eran descripciones sin forma de verificarlas.	Se convierten en 10 requisitos funcionales con criterio de aceptación y trazabilidad hacia F1–F12.	2
5	Los servicios eran una lista sin responsabilidades ni relaciones.	Catálogo con responsabilidad, datos propios y dependencias de cada servicio, más mapa de comunicación.	6, 7
6	Tecnologías «preliminares» con alternativas abiertas (Node.js o Spring Boot, PostgreSQL o MySQL).	Se decide Node.js con Express y PostgreSQL, con justificación y alternativas descartadas.	9

#	Hallazgo en la Entrega 1	Corrección aplicada	Sección
7	Las reglas de préstamo no tenían valores.	Se definen 9 reglas iniciales con valores propuestos y configurables, por validar con un encargado real.	2.3
8	No se indicaba de dónde obtienen datos los reportes.	Reportes es solo lectura y consume a los demás servicios por API; no accede a sus bases de datos.	6, 7
9	Las reglas de préstamo dependían de una entrevista y de registros reales que no se habían obtenido.	Se documentan como supuestos del equipo (sección 12) y las reglas de negocio se validarán con un encargado de laboratorio en la semana 3.	12

1.2 Alcance revisado de la primera versión

- Autenticación y roles (alumno, docente, encargado, administrador).
- Catálogo de laboratorios, categorías y recursos con estado administrativo.
- Consulta de disponibilidad y reserva con validación de traslapes y reglas de préstamo.
- Entrega y devolución con estado del recurso, incidencias e historial.
- Sanción temporal automática por retraso (sin cobro económico).
- Notificaciones registradas y enviadas por correo de prueba.
- Reportes de uso, retrasos e incidencias.

Sigue fuera de alcance: aplicación móvil, integración con sistemas escolares, cobro de multas, mantenimiento preventivo, QR por hardware e interfaz web completa. La API se consumirá desde Postman y Swagger UI.

2. Requisitos funcionales

Cada requisito indica qué actor lo usa, qué servicio lo implementa, su prioridad para la primera versión (Alta, Media o Baja) y un criterio con el que se podrá comprobar que está cumplido.

2.1 Tabla de requisitos funcionales

ID	Requisito	Actor	Servicio	Prior.	Criterio de aceptación
RF-01	Registrar usuarios e iniciar sesión con token de acceso.	Todos	Usuarios	Alta	Con datos válidos se crea el usuario con contraseña cifrada y el inicio de sesión devuelve un token con expiración. Correo repetido: 409. Credenciales inválidas: 401.

ID	Requisito	Actor	Servicio	Prior.	Criterio de aceptación
RF-02	Cambiar el rol y el estado de cuenta de un usuario.	Admin.	Usuarios	Alta	Solo el administrador puede hacerlo; otro rol recibe 403.
RF-03	Gestionar laboratorios, categorías y recursos, incluido su estado (activo, mantenimiento, baja).	Encargado, admin.	Inventario	Alta	El código de inventario es único. Un recurso en mantenimiento o baja rechaza nuevas reservas.
RF-04	Consultar disponibilidad por tipo, laboratorio, fecha y horario.	Alumno, docente, encargado	Reservas	Alta	Devuelve solo recursos activos sin traslape en el horario pedido, con filtros y paginación.
RF-05	Crear, cancelar o modificar reservas aplicando las reglas de préstamo (límites y sanciones).	Alumno, docente; encargado	Reservas, Usuarios	Alta	Reserva válida: 201. Horario traslapado: 409. Usuario sancionado o sobre su límite: rechazo con mensaje claro.
RF-06	Registrar la entrega y la devolución del recurso con su estado.	Encargado	Reservas	Alta	La entrega exige una reserva confirmada y vigente. Una devolución «dañado» pone el recurso en mantenimiento.
RF-07	Registrar incidencias (daño, faltante, retraso) y sancionar con bloqueo temporal los retrasos.	Encargado, admin.; sistema	Reservas, Usuarios	Media	Cada incidencia queda ligada a un préstamo. Un retraso fuera de tolerancia crea una sanción con fecha de fin que impide reservar.
RF-08	Notificar al usuario (confirmación, recordatorio, retraso) y permitirle consultar sus avisos.	Sistema; alumno, docente	Notificaciones	Media	Tras crear una reserva existe una notificación registrada. Si Notificaciones falla, la reserva se crea igual.
RF-09	Consultar el historial de reservas y préstamos por usuario, recurso y periodo.	Alumno, docente (propio); encargado; admin.	Reservas	Media	Resultado filtrado y paginado. Un alumno que pide el historial de otro recibe 403.
RF-10	Generar reportes de uso, ocupación, retrasos e incidencias por periodo.	Encargado, admin.	Reportes	Media	Los totales coinciden con los registros de Reservas del mismo periodo.

2.2 Trazabilidad con las funciones de la Entrega 1

Función (Entrega 1)	Requisitos que la cubren
F1 Registro e inicio de sesión	RF-01
F2 Roles y permisos	RF-02 y la matriz de la sección 4
F3 Catálogo	RF-03

Función (Entrega 1)	Requisitos que la cubren
F4 Consulta de disponibilidad	RF-04
F5 Crear reserva	RF-05
F6 Cancelar o modificar reserva	RF-05
F7 y F8 Entrega y devolución	RF-06
F9 Historial	RF-09
F10 Notificaciones	RF-08
F11 Incidencias	RF-07
F12 Reportes de uso	RF-10
Nuevo en esta entrega	Sanción automática por retraso (dentro de RF-07), que faltaba para cumplir las reglas de préstamo

2.3 Reglas de negocio iniciales

Los valores numéricos son propuestas del equipo, configurables, y deben validarse con un encargado de laboratorio.

ID	Regla	Requisito relacionado
RN-01	Un recurso no puede tener dos reservas activas con horarios traslapados.	RF-05, RNF-05
RN-02	Solo los recursos en estado activo pueden reservarse.	RF-03, RF-05
RN-03	Límite de reservas vigentes: alumno 2, docente 5 (valores propuestos).	RF-05
RN-04	Un usuario con sanción vigente no puede crear reservas.	RF-05, RF-07
RN-05	Una reserva puede cancelarse o modificarse hasta 1 hora antes de su inicio (valor propuesto).	RF-05
RN-06	Una devolución posterior al fin de la reserva más 15 minutos de tolerancia (valor propuesto) genera una incidencia de retraso y una sanción temporal.	RF-06, RF-07
RN-07	Una devolución con daño pone el recurso en mantenimiento y crea una incidencia.	RF-06, RF-07
RN-08	Solo el encargado del laboratorio del recurso, o el administrador, registra entregas y devoluciones.	RF-06

ID	Regla	Requisito relacionado
RN-09	La entrega requiere una reserva confirmada y vigente; en mostrador, el encargado crea la reserva y la entrega en el mismo momento.	RF-05, RF-06

3. Requisitos no funcionales

Las metas numéricas son valores iniciales para medir el avance; se ajustarán con las pruebas de la Entrega 7.

ID	Categoría	Requisito	Cómo se verificará
RNF-01	Seguridad: credenciales	Las contraseñas se guardan con hash bcrypt. Los tokens tienen expiración (1 hora propuesta). No hay secretos en el repositorio.	Revisar la tabla de usuarios sin texto plano; buscar secretos en el repositorio; existe un archivo .env.example.
RNF-02	Seguridad: autorización	Toda ruta protegida valida el token y el rol. Sin token responde 401; con rol insuficiente, 403.	Casos negativos por cada rol en Postman y en pruebas automáticas.
RNF-03	Rendimiento	La consulta de disponibilidad responde en menos de 500 ms (percentil 95) con 50 usuarios concurrentes en entorno local.	Prueba de carga con autocannon en la Entrega 7.
RNF-04	Tolerancia a fallos	Si Notificaciones o Reportes dejan de funcionar, se puede seguir reservando, entregando y devolviendo.	Detener esos contenedores y ejecutar el flujo completo de reserva.
RNF-05	Integridad de datos	Dos reservas traslapadas del mismo recurso nunca coexisten, ni con peticiones simultáneas. Se garantiza con una restricción en la base de datos y transacciones.	Enviar dos peticiones simultáneas para el mismo horario: una responde 409.
RNF-06	Interoperabilidad y usabilidad de la API	API REST en JSON con rutas versionadas (/api/v1), formato único de errores y documentación OpenAPI.	Swagger UI accesible y contratos revisados en la Entrega 3.
RNF-07	Mantenibilidad	Cada servicio se organiza en rutas, controladores, servicios y acceso a datos. La lógica crítica tiene pruebas automáticas con meta de cobertura del 60 %.	Revisión de la estructura del repositorio y reporte de cobertura.
RNF-08	Portabilidad y despliegue	Todo el sistema levanta con un solo comando (docker compose up), configurado con variables de entorno.	Clonar en una máquina limpia y seguir el README.
RNF-09	Trazabilidad	Cada entrega, devolución e incidencia registra quién la hizo, cuándo y sobre qué recurso.	Consultar los registros en la base de datos y en el historial.

4. Actores, roles y permisos iniciales

4.1 Actores

Actor	Descripción	Qué necesita hacer
Alumno	Estudiante que usa equipo y espacios para prácticas y proyectos.	Consultar disponibilidad, reservar y cancelar sus reservas, ver su historial y sus avisos.
Docente	Profesor que reserva equipo o laboratorios para clases y prácticas.	Lo mismo que el alumno, con un límite de reservas mayor.
Encargado de laboratorio	Persona responsable del equipo de uno o más laboratorios.	Administrar el catálogo de su laboratorio, entregar y recibir equipo, registrar incidencias y ver reportes de su laboratorio.
Administrador	Responsable del área o coordinación.	Gestionar usuarios y roles, laboratorios y categorías, y ver todos los reportes.
Sistema (actor no humano)	Tareas programadas y reglas automáticas.	Detectar retrasos, aplicar sanciones y generar notificaciones.

4.2 Matriz de permisos

«Propias» significa solo sobre sus propios registros; «Su lab.» significa solo sobre los laboratorios que tiene a cargo.

Operación	Alumno	Docente	Encargado	Admin.	Servicio
Registrarse (sin sesión)	Sí	Sí	—	—	Usuarios
Iniciar sesión	Sí	Sí	Sí	Sí	Usuarios
Cambiar rol o estado de un usuario	—	—	—	Sí	Usuarios
Consultar catálogo y disponibilidad	Sí	Sí	Sí	Sí	Inventario, Reservas
Crear, editar o dar de baja recursos	—	—	Su lab.	Sí	Inventario
Gestionar laboratorios y categorías	—	—	—	Sí	Inventario
Cambiar estado de un recurso	—	—	Su lab.	Sí	Inventario
Crear reserva	Sí	Sí	Sí (mostrador)	—	Reservas

Operación	Alumno	Docente	Encargado	Admin.	Servicio
Cancelar o modificar reserva	Propias	Propias	Su lab.	Sí	Reservas
Registrar entrega y devolución	—	—	Su lab.	Sí	Reservas
Registrar incidencia	—	—	Su lab.	Sí	Reservas
Ver historial de préstamos	Propio	Propio	Su lab.	Sí	Reservas
Ver notificaciones	Propias	Propias	Propias	Propias	Notificaciones
Ver reportes	—	—	Su lab.	Sí	Reportes

5. Entidades principales del dominio

Cada entidad pertenece a un solo servicio. Las referencias entre servicios (por ejemplo, usuario_id dentro de una reserva) son identificadores lógicos: no habrá llaves foráneas entre bases de datos distintas. El modelo entidad-relación completo se elabora en la Entrega 4.

Entidad	Descripción	Atributos principales	Servicio dueño
Usuario	Persona que usa el sistema.	id, nombre, correo (único), matrícula o clave, contraseña cifrada, rol, estado (activo o bloqueado)	Usuarios
Rol	Nivel de permisos.	id, nombre (alumno, docente, encargado, administrador)	Usuarios
Sanción	Bloqueo temporal de un usuario.	id, usuario, motivo, fecha de inicio, fecha de fin, préstamo que la originó	Usuarios
Laboratorio	Espacio físico con equipo a cargo.	id, nombre, ubicación, encargado (referencia a usuario)	Inventario
Categoría	Clasificación de recursos.	id, nombre, tipo (equipo o espacio)	Inventario
Recurso	Equipo o espacio prestable.	id, nombre, código de inventario (único), categoría, laboratorio, estado (activo, mantenimiento, baja)	Inventario
Reserva	Solicitud de un recurso en un horario.	id, usuario, recurso, inicio, fin, estado (confirmada, cancelada, en curso, finalizada)	Reservas
Préstamo	Entrega y devolución real del recurso.	id, reserva, entregado por, fecha de entrega, recibido por, fecha de devolución, estado de devolución (correcto, dañado, incompleto), observaciones	Reservas
Incidencia	Daño, faltante o retraso.	id, préstamo, recurso, tipo, descripción, fecha, estado	Reservas

Entidad	Descripción	Atributos principales	Servicio dueño
Notificación	Aviso enviado a un usuario.	id, usuario, correo de destino, tipo, mensaje, estado de envío, fecha	Notificaciones

Relaciones principales: un laboratorio tiene varios recursos; una categoría agrupa varios recursos; un usuario tiene varias reservas; una reserva tiene como máximo un préstamo; un préstamo puede originar incidencias y, si hay retraso, una sanción.

6. Servicios y módulos principales

La solución se compone de un API Gateway y cinco servicios. Cada servicio es dueño de sus datos y solo se accede a ellos mediante su API.

6.1 API Gateway

Responsabilidad	Único punto de entrada. Enruta las peticiones al servicio correspondiente, valida el token JWT, aplica CORS y limita la cantidad de peticiones. Reenvía la identidad y el rol del usuario a los servicios.
No es responsable de	Lógica de negocio ni acceso a datos.
Datos propios	Ninguno.
Requisitos	RNF-01, RNF-02, RNF-06

6.2 Usuarios y Autenticación

Responsabilidad	Registro, inicio de sesión, emisión de tokens, gestión de usuarios y roles, y sanciones con el estado de cuenta.
No es responsable de	Reservas, recursos ni avisos.
Datos propios	Usuario, Rol, Sanción.
Requisitos	RF-01, RF-02, RF-07
Consume a	Ningún servicio.

6.3 Inventario

Responsabilidad	Catálogo de laboratorios, categorías y recursos, y el estado administrativo de cada recurso (activo, mantenimiento, baja).
No es responsable de	Saber si un recurso está reservado o prestado: eso lo calcula Reservas.

Datos propios	Laboratorio, Categoría, Recurso.
Requisitos	RF-03
Consume a	Ningún servicio.

6.4 Reservas y Préstamos

Responsabilidad	Núcleo del negocio: disponibilidad por horario, reservas, reglas de préstamo, entrega y devolución, incidencias e historial. Ejecuta la tarea programada que detecta retrasos.
No es responsable de	Credenciales, catálogo de recursos ni envío de avisos.
Datos propios	Reserva, Préstamo, Incidencia.
Requisitos	RF-04 a RF-07, RF-09
Consume a	Usuarios, Inventario y Notificaciones.

6.5 Notificaciones

Responsabilidad	Registrar los avisos y enviarlos por correo de prueba o simulado. Recibe el correo de destino en la petición, por lo que no depende de otros servicios.
No es responsable de	Decidir cuándo se avisa: lo decide el servicio que la invoca.
Datos propios	Notificación.
Requisitos	RF-08
Consume a	Ningún servicio.

6.6 Reportes

Responsabilidad	Reportes de uso, ocupación, retrasos e incidencias, calculados a partir de los datos de otros servicios.
No es responsable de	Modificar datos ni guardar información propia.
Datos propios	Ninguno (solo lectura).
Requisitos	RF-10
Consume a	Reservas, Inventario y Usuarios.

7. Comunicación entre servicios

Los servicios se comunican por HTTP con mensajes JSON de forma síncrona. Los números corresponden al diagrama de la sección 8. Ningún servicio lee la base de datos de otro.

#	Origen → destino	Para qué	Si el destino falla
—	Gateway → todos los servicios	Enrutar la petición ya autenticada.	Responde 503 al cliente.
①	Reportes → Reservas	Leer reservas, préstamos e incidencias (paginado) para agregarlos.	El reporte responde 503; la operación diaria no se afecta.
②	Reportes → Inventario	Leer nombres de recursos, categorías y laboratorios.	El reporte responde 503.
③	Reportes → Usuarios	Leer nombre y rol para reportes por usuario.	El reporte responde 503.
④	Reservas → Inventario	Verificar que el recurso existe y está activo al reservar. Poner el recurso en mantenimiento cuando la devolución llega dañada.	La operación se rechaza con 503. Se prioriza la consistencia sobre la disponibilidad.
⑤	Reservas → Usuarios	Verificar estado de cuenta y obtener el correo al reservar. Crear la sanción cuando hay retraso.	La operación se rechaza con 503.
⑥	Reservas → Notificaciones	Registrar avisos de confirmación, entrega, recordatorio y retraso.	No bloquea: se escribe en el registro de eventos y se reintenta después (RNF-04).

7.1 Ejemplo: crear una reserva

Paso	Qué ocurre
1	El alumno envía POST /api/v1/reservas con su token. El Gateway valida la firma y la vigencia del token y reenvía la petición con la identidad y el rol del usuario.
2	Reservas valida el cuerpo de la petición (recurso, inicio y fin, con fin posterior al inicio).
3	Reservas consulta a Usuarios (⑤): cuenta activa, sin sanción vigente, y correo del usuario.
4	Reservas consulta a Inventario (④): el recurso existe y está activo.
5	Reservas verifica el límite de reservas vigentes del rol (RN-03) e inserta la reserva en una transacción. La base de datos rechaza cualquier traslape (RN-01).
6	Reservas envía la confirmación a Notificaciones (⑥) sin esperar la respuesta.
7	Reservas responde 201 con la reserva creada.
Errores previstos	401 sin token válido; 403 rol sin permiso; 404 recurso inexistente; 409 horario traslapado; 422 regla de negocio incumplida; 503 servicio dependiente no disponible. Los códigos definitivos se fijan en la Entrega 3.

8. Diagrama de arquitectura SOA

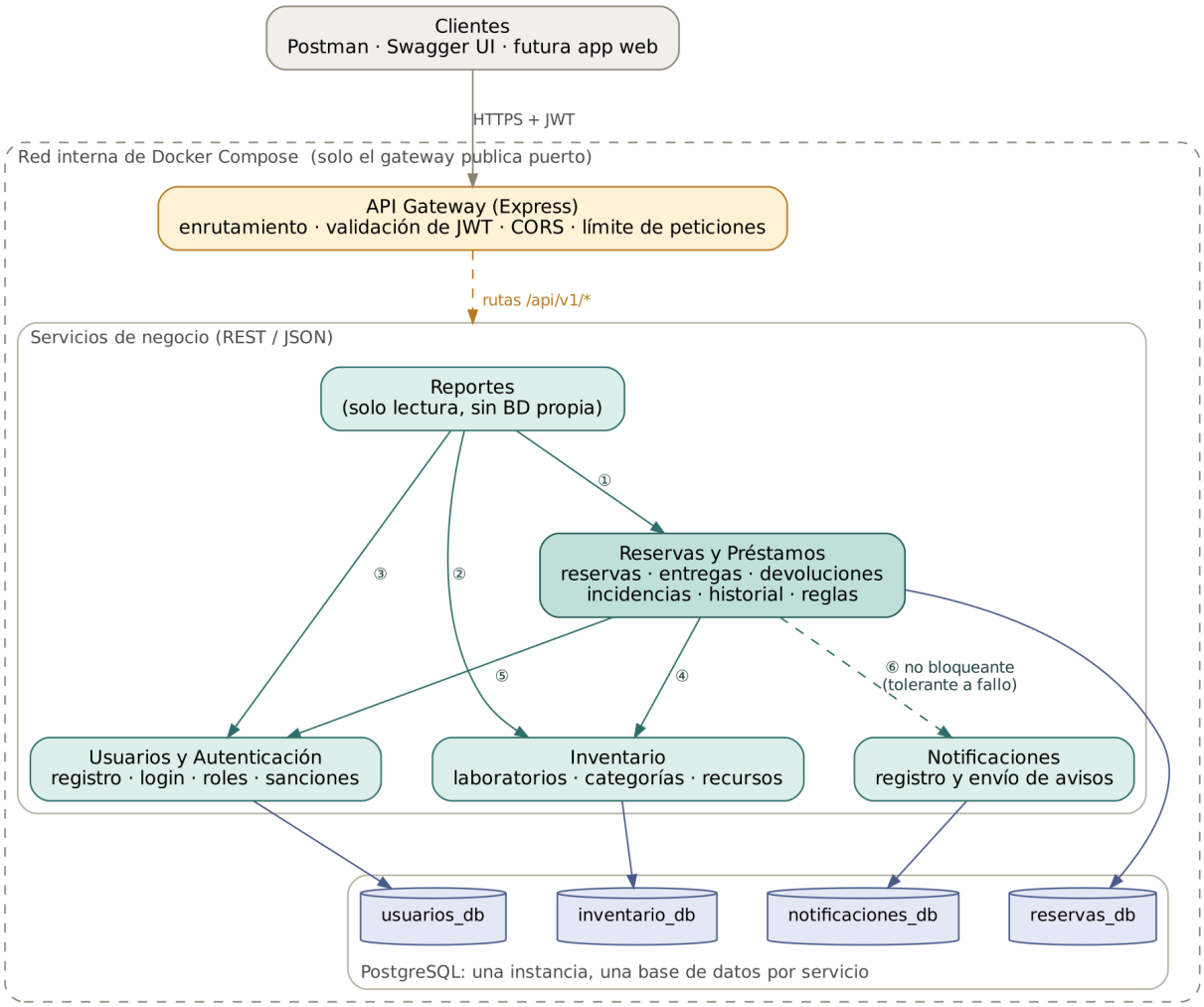


Figura 1. Arquitectura SOA de PrestaLab. Las flechas continuas son llamadas REST entre servicios; la línea punteada ⑥ no bloquea la operación.

Solo el Gateway publica un puerto hacia el exterior. Los servicios y la base de datos viven en la red interna de Docker Compose. Se usa una sola instancia de PostgreSQL con una base de datos independiente por servicio para simplificar el despliegue sin perder la separación de datos.

9. Stack tecnológico propuesto y justificación

Estas son las decisiones del equipo para el resto del proyecto. Cada una indica por qué se elige y qué alternativa se descartó.

Capa	Tecnología	Justificación	Alternativa descartada
Lenguaje y framework	Node.js con Express	Un solo lenguaje para el gateway y los servicios. Permite construir APIs REST con poco código base y hay abundante documentación para el equipo.	Spring Boot: más configuración y curva de aprendizaje para el tiempo disponible.
Base de datos	PostgreSQL	Soporta restricciones de exclusión sobre rangos de tiempo, que impiden por sí mismas dos reservas traslapadas (RNF-05). Ofrece transacciones, llaves foráneas y CHECK.	MySQL: no ofrece una restricción equivalente y obligaría a validar el traslape solo en código.
Acceso a datos	Librería pg con consultas SQL parametrizadas y scripts SQL de migración	Mantiene visibles las restricciones y los scripts que pide la Entrega 4, y evita consultas construidas con texto.	ORM (Sequelize, Prisma): oculta el SQL y complica las restricciones de exclusión.
Autenticación	JWT y bcrypt	Los tokens son autocontenidos, así que el Gateway los valida sin consultar a Usuarios. bcrypt cumple RNF-01.	Sesiones en servidor: requieren estado compartido entre servicios. Keycloak u OAuth2 completo: excesivo para el alcance.
Validación	Zod (o Joi)	Esquemas declarativos para validar cuerpos y parámetros, con mensajes de error uniformes.	Validar a mano: repetitivo y propenso a omisiones.
Comunicación	REST con JSON, de forma síncrona	Simple de probar con Postman y de documentar con OpenAPI. Cumple lo que pide el curso.	Broker de mensajes (RabbitMQ, Kafka): se pospone como mejora futura para Notificaciones.
Gateway	Express con http-proxy-middleware	Misma tecnología que el resto; permite validar JWT y CORS en código propio.	Nginx o Kong: añaden otra herramienta y configuración.
Documentación	OpenAPI 3 con Swagger UI	Contratos legibles sin leer el código, como pide la Entrega 7.	Documento manual: se desactualiza.
Pruebas	Jest con Supertest; autocannon para carga	Cubren pruebas de endpoints, casos negativos y la prueba básica de carga.	Pruebas solo manuales: no son repetibles.
Despliegue	Docker y Docker Compose	Levanta todos los servicios y la base de datos con un comando (RNF-08).	Instalación manual: no es reproducible.
Control de versiones	Git y GitHub	Una rama por funcionalidad, tag entrega-N en cada semana.	—
Configuración	Variables de entorno (.env) y .env.example	Separa los secretos del código (RNF-01).	Valores fijos en el código.

10. Justificación de la división en servicios

- Cohesión por dominio: cada servicio agrupa datos y reglas que cambian por la misma razón (identidad y seguridad, catálogo físico, ciclo de vida del préstamo, avisos, análisis).
- Un solo dueño por dato: ningún servicio accede a la base de otro. Así, «disponibilidad» no se duplica: Inventario sabe si un recurso puede usarse; Reservas sabe si está libre en un horario.
- Aislamiento de fallos: Notificaciones y Reportes no son críticos para la operación, mientras que Usuarios e Inventario sí lo son.
- Fronteras de seguridad: las credenciales solo existen en Usuarios.
- Tamaño manejable: cinco servicios más el Gateway permiten repartir el trabajo entre 3 a 5 integrantes en 8 semanas.

Alternativas consideradas

Alternativa	Por qué se descartó
Un solo sistema (monolito)	No demuestra una arquitectura orientada a servicios y mezcla el despliegue y las reglas de todos los módulos.
Un servicio por entidad (unos 10)	Genera demasiada comunicación y operación para un equipo pequeño; muchas entidades cambian juntas.
Separar Reservas y Préstamos	Comparten ciclo de vida y transacción. Se mantienen unidos con módulos internos separados y se podrán dividir si el servicio crece.

Limitaciones conocidas

- Reservas concentra la mayor parte de la lógica y puede volverse un servicio grande.
- Reportes consulta por API y agrega en memoria; con muchos más datos o usuarios convendría una base de lectura o eventos.
- La comunicación síncrona acopla la disponibilidad de Reservas a Usuarios e Inventario.
- El Gateway valida el token y los servicios confían en él; por eso solo el Gateway debe ser accesible desde fuera, y se revisará en la Entrega 7.

11. Verificación de entregables y criterios

11.1 Trabajo requerido

Trabajo requerido	Dónde se cumple
Revisar y corregir la propuesta de la semana 1	Sección 1
Identificar requisitos funcionales	Sección 2

Trabajo requerido	Dónde se cumple
Identificar requisitos no funcionales	Sección 3
Definir actores y permisos iniciales	Sección 4
Identificar entidades principales del dominio	Sección 5
Identificar servicios o módulos principales	Sección 6
Definir la responsabilidad de cada servicio	Sección 6 (cada servicio indica qué hace y qué no)
Indicar qué servicios se comunican entre sí	Sección 7
Proponer el stack tecnológico	Sección 9
Crear un diagrama de arquitectura inicial	Sección 8, Figura 1

11.2 Entregables obligatorios y criterios de aceptación

Entregable obligatorio	Dónde se cumple
Documento actualizado con requisitos	Secciones 1 a 3
Tabla de requisitos funcionales y no funcionales	Secciones 2.1 y 3 (10 RF y 9 RNF; mínimo pedido: 10 y 5)
Lista de actores/roles	Sección 4 (4 roles; mínimo pedido: 2)
Catálogo de servicios con responsabilidad de cada uno	Sección 6 (5 servicios y Gateway; mínimo pedido: 4)
Diagrama de arquitectura SOA	Sección 8, Figura 1
Justificación del stack tecnológico	Sección 9

Criterio de aceptación	Cómo se cumple
Servicios con responsabilidades claras y no duplicadas	Cada servicio lista lo que sí y lo que no hace; cada dato tiene un solo dueño (secciones 5 y 6).
La arquitectura corresponde al problema	Cada requisito apunta a un servicio; los servicios salen del flujo reservar, entregar, devolver.
Los requisitos pueden verificarse	RF y RNF incluyen un criterio o forma de verificación.
El equipo puede explicar la división	Sección 10.

12. Fuentes y referencias

- Proyecto_SOA.pdf: lineamientos del Proyecto Integrador, Entrega 2.

- Proceso actual de préstamo descrito en la Entrega 1. Los valores de las reglas de negocio (RN-01 a RN-09) son supuestos del equipo, pendientes de validar con un encargado de laboratorio.
- Erl, T. (2005). Service-Oriented Architecture: Concepts, Technology, and Design. Prentice Hall.
- Newman, S. (2021). Building Microservices, 2.ª ed. O'Reilly Media.
- Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (tesis doctoral). University of California, Irvine.
- Jones, M., Bradley, J. y Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. https://www.rfc-editor.org/rfc/rfc7519
- The PostgreSQL Global Development Group. PostgreSQL Documentation: Exclusion Constraints. https://www.postgresql.org/docs/current/ddl-constraints.html
- OpenAPI Initiative. OpenAPI Specification. https://spec.openapis.org

Anexo A. Bitácora de cambios

Semana	Cambio realizado	Motivo	Responsable(s)	Evidencia
1	Versión inicial de la propuesta	Primera entrega	Equipo	Documento de la Entrega 1
2	Estado administrativo separado de la disponibilidad	Evitar responsabilidades duplicadas entre Inventario y Reservas	Equipo	Secciones 1.1 y 6
2	Definición de sanción y alcance de notificaciones	Eran ambiguas en la propuesta	Equipo	Secciones 1.1 y 2.3
2	Decisión de stack (Node.js, Express, PostgreSQL)	Dejar de tener alternativas abiertas	Equipo	Sección 9
2	10 RF, 9 RNF, matriz de permisos, servicios y diagrama	Contenido de la Entrega 2	Equipo	Este documento